

Learned Communication Under Heartbeat Delay and Teammate Dropout in Cooperative Multi-Agent Reinforcement Learning

Praneel Khiantani

School of Engineering and Applied Sciences
Harvard University
Cambridge, MA
praneelkhiantani@college.harvard.edu

Sam Chen

School of Engineering and Applied Sciences
Harvard University
Cambridge, MA
sam_chen@college.harvard.edu

Abstract—We study cooperative multi-agent reinforcement learning (MARL) under partial observability of teammate liveness, a setting inspired by distributed-systems heartbeat protocols. In our framework, agents receive delayed heartbeat signals from teammates, and one agent may permanently drop out mid-episode. Because heartbeats already in flight still arrive after a teammate dies, a receiving agent faces a short ambiguity window where it cannot distinguish “dead” from “merely slow.” We build novel infrastructure on top of the Robotic Warehouse (RWARE) environment to support configurable heartbeat delay, dropout schedules, and a discrete communication channel. We extend Multi-Agent PPO (MAPPO) with a RIAL-style learned message head (Comm-MAPPO) and evaluate whether communication rescues performance under dropout. Our results show that naive policy-gradient-trained discrete messages remain largely ungrounded, failing to improve unshaped evaluation returns despite boosting shaped training returns. We then introduce GComm-MAPPO, which adds a cross-entropy auxiliary loss that grounds messages to task-relevant intent. Under targeted request-intent dropout, GComm-MAPPO significantly outperforms both no-communication and ungrounded communication baselines ($p < 0.01$, Cohen’s $d_z > 1.3$). Our findings demonstrate that message grounding, not channel capacity, is the binding constraint for communication-aided dropout robustness in cooperative MARL.

Index Terms—multi-agent reinforcement learning, communication, dropout robustness, MAPPO, RIAL, cooperative AI

I. Introduction

In cooperative multi-agent systems, agents must coordinate effectively even when teammates may fail unexpectedly. This challenge arises in warehouse robotics, search-and-rescue, and autonomous vehicle platoons, where an agent’s sudden departure forces survivors to infer what happened and adapt their behavior. The problem is compounded under partial observability: if agents receive only delayed liveness signals, distinguishing a crashed teammate from a merely slow one becomes nontrivial.

We draw a direct analogy to distributed-systems heartbeat protocols, where a missing heartbeat can indicate either node failure or network delay. In cooperative MARL, we formalize this as liveness ambiguity: agents observe

heartbeat-derived features that lag by D steps, and in-flight heartbeats still arrive after a sender dies, creating a bounded window of uncertainty.

Our central research question is: Can learned inter-agent communication help agents disambiguate true teammate dropout from stale observations and thereby improve cooperative task performance? We frame this as an interaction effect using a difference-in-differences design: communication should help more when dropout creates hidden-state ambiguity (delay-dropout regime) than when only delay is present (delay-only regime).

This work falls under the CS 2860 topic of multi-agent reinforcement learning with communication. We make three contributions beyond our MAPPO baseline: (1) a novel communication-and-dropout infrastructure layer built on RWARE, (2) Comm-MAPPO, a RIAL-style extension of MAPPO with a discrete message channel, and (3) GComm-MAPPO, an intent-grounded variant that uses a cross-entropy auxiliary loss to anchor messages to task-relevant semantics. Our results reveal that grounding—not channel capacity—is the critical bottleneck for communication-aided robustness.

II. Literature Review

Multi-Agent PPO (MAPPO). Yu et al. [1] demonstrate that PPO with a centralized value function achieves strong performance in cooperative MARL under the Centralized Training with Decentralized Execution (CTDE) paradigm. MAPPO uses parameter sharing across agents and a centralized critic that conditions on global state during training. However, the standard MAPPO framework assumes all agents remain active throughout an episode and does not include an explicit inter-agent communication channel—two gaps our work addresses.

Learned Communication (RIAL/DIAL). Foerster et al. [2] introduce Reinforced Inter-Agent Learning (RIAL) and Differentiable Inter-Agent Learning (DIAL), two paradigms for emergent communication. In RIAL, message selection is treated as an additional discrete action trained via policy gradients, while DIAL passes continuous

gradients through the communication channel. A known limitation of RIAL is that policy-gradient training provides only sparse reward signal for message selection, often leaving messages ungrounded [8]. Our Comm-MAPPO extends MAPPO with a RIAL-style message head, and our experiments confirm this grounding failure, motivating our intent-auxiliary approach.

Hierarchical MARL with Communication. Ghavamzadeh, Mahadevan, and Makar [3] propose cooperative hierarchical reinforcement learning (HRL) where agents communicate at subtask boundaries. High-level policies select temporally extended subtasks while low-level controllers execute primitive actions. We implemented a faithful cooperative HRL baseline with SMDP Q-learning and a RECOVER subtask for dropout scenarios. While the hierarchical communication concept is appealing, our experiments showed near-zero absolute returns due to low-level execution bottlenecks, confirming that HRL requires stronger primitive controllers before it can compete with flat MAPPO in RWARE.

Gaps in Adjacent Work. Existing MARL communication methods (CommNet [5], TarMAC [6], QMIX [7]) assume all agents persist throughout episodes and do not model teammate failure. Dropout robustness work in MARL [9] typically does not combine delayed observations with learned communication. Our work uniquely combines heartbeat-based partial observability, permanent mid-episode dropout, and learned communication within a unified experimental framework, enabling controlled study of each factor’s contribution.

III. Baseline: MAPPO

Our baseline algorithm is Multi-Agent Proximal Policy Optimization (MAPPO) [1], implemented under the CTDE paradigm.

A. Algorithm Description

Each of N agents shares a common actor network π_θ that maps local observations o_i plus a one-hot agent identifier to a distribution over environment actions. A centralized critic V_ϕ takes the concatenation of all agents’ observations and an alive-bit mask as input and outputs a scalar state value. During training, the critic has access to global information including the true alive status of each agent; during execution, agents act based solely on local observations.

The policy is updated using PPO’s clipped surrogate objective. We use Generalized Advantage Estimation (GAE) with $\gamma = 0.99$ and $\lambda = 0.95$ over a team reward signal (sum of individual rewards for alive agents). Key hyperparameters include: learning rates $\alpha_{\text{actor}} = 3 \times 10^{-4}$, $\alpha_{\text{critic}} = 1 \times 10^{-3}$; clip range $\epsilon = 0.2$; entropy coefficient 0.01; 4 update epochs with 4 minibatches per epoch; two-layer MLPs with 128 hidden units.

B. Environment: Robotic Warehouse (RWARE)

We evaluate on RWARE [4], a grid-world cooperative task where agents must pick up requested shelves and deliver them to designated goal locations. The action space is {noop, forward, left, right, toggle_load}. Agents receive partial observations including their position, orientation, carried shelf status, and nearby shelf/agent information. A delivery (shelf reaching the goal) yields +1 reward. We additionally use reward shaping with a +0.5 pickup bonus during training to mitigate reward sparsity, while evaluation uses only unshaped delivery rewards.

C. What Is Missing from the Baseline

Standard MAPPO in RWARE has three critical gaps that motivate our extensions: (1) agents have no mechanism to communicate intent or status beyond implicit coordination through shared training; (2) the environment assumes all agents are always active, with no support for mid-episode failure; and (3) observations are instantaneous, ignoring the realistic scenario where liveness information arrives with delay. These gaps mean MAPPO cannot distinguish between a teammate that has failed and one that is simply in a different part of the warehouse.

IV. Extended Results Study

We present three extensions beyond baseline MAPPO, each requiring substantial novel code. All experiments use the matrix runner infrastructure we built, with cross-machine seed pooling for reproducibility. We report both shaped training returns and unshaped evaluation returns, highlighting when they disagree.

A. Extension 1: Communication-Dropout Infrastructure for RWARE

Our first contribution is a modular infrastructure layer that adds configurable heartbeat delay, teammate dropout, and a discrete communication channel to any RWARE environment. This required approximately 1,200 lines of novel wrapper code across four modules: unified.py (environment wrapper with dropout logic and observation management), heartbeat.py (heartbeat tracker), rware_adapter.py (RWARE-specific interface), and run_rware_matrix.py (automated experiment runner).

1) **Unified Environment Wrapper:** The UnifiedMARLEnv wrapper normalizes heterogeneous environment APIs into a fixed-shape contract: at each step, it accepts an $(N, 2)$ action array (environment action + message token) and returns observations of shape (N, d_{obs}) , an alive mask, available-action masks, and debug metadata. The wrapper handles dead-agent masking internally: upon dropout, the failed agent’s environment actions are forced to noop, its rewards are zeroed, and its observations are frozen to the last live snapshot. The wrapper also appends heartbeat-derived freshness features and received message tokens to each agent’s observation vector, producing a self-contained input suitable for any flat MARL algorithm.

2) Heartbeat Tracker: We implemented a Heartbeat-Tracker that models realistic liveness signaling inspired by distributed-systems failure detectors. Each alive agent emits a heartbeat every P steps (default $P=1$); delivery is delayed by D steps (we primarily test $D=30$). The tracker maintains per-pair age counters age_{ij} tracking how many steps since agent i last received a heartbeat from agent j . Normalized freshness features $f_{ij} = 1 - \min(\text{age}_{ij}, C)/C$ (where C is a clip constant, default $C=128$) are appended to each agent’s observation. Critically, heartbeats already in transit when an agent dies continue to arrive on schedule, creating a bounded ambiguity window of up to D steps where a dead agent’s heartbeat appears fresh. This mirrors real distributed systems where in-flight TCP packets arrive after node failure.

3) Dropout Mechanism: We support three dropout modes: (a) fixed: a predetermined agent dies at a fixed timestep; (b) window: a random agent dies at a uniformly sampled time $t \sim \text{Uniform}[t_{\text{start}}, t_{\text{end}}]$ (we use $[200, 350]$ for 4-agent experiments); and (c) targeted request-intent: the agent with the highest request-relevance score is selected for dropout at runtime, maximizing the coordination disruption. The dropout is permanent—once an agent dies, it never returns. The critic receives the true alive mask (CTDE), but the actor does not—preserving the partial observability that makes communication potentially valuable.

4) Message Echo for Ambiguity Preservation: A subtle but critical design choice concerns what dead agents “say.” If dead agents emit all-zero messages, the communication channel trivially leaks an oracle death signal, bypassing the heartbeat ambiguity we aim to study. Instead, dead agents echo their last emitted one-hot message token indefinitely. Messages are initialized to token-0 one-hot vectors at episode reset. This preserves liveness ambiguity through the communication channel and ensures any communication benefit must come from learned message semantics rather than a trivial death indicator. We validate this design choice with a dedicated oracle diagnostic in Extension 2.

5) Experimental Matrix Runner: We built an automated experiment runner that sweeps the cross-product of methods $\times$ regimes $\times$ heartbeat delays $\times$ seeds. The runner supports configurable parallelism ($-\text{max-parallel}$), per-cell thread capping to prevent CPU oversubscription, and clean signal handling. Production evaluation mode ($-\text{production-eval}$) uses 30 evaluation episodes every 25 training updates, reducing eval noise by $\sqrt{10}\times$ compared to our initial 3-episode protocol. Results are pooled across machines using a verification script that aborts on any config mismatch between source runs.

6) Results: Dropout Headroom Analysis: Before measuring communication benefit, we must verify that dropout actually hurts baseline performance—otherwise there is no headroom for communication to rescue. Table I shows that dropout’s impact varies strongly with

TABLE I: Dropout headroom: eval return difference (delay-dropout minus delay-only) under heartbeat-only MAPPO. Negative values indicate dropout hurts.

Environment	Agents	Δ Eval	Status
rware-tiny-4ag-v2	4	+11.4	Confounded
rware-small-4ag-v2	4	+11.8	Confounded
rware-medium-4ag-v2	4	−2.0	Clean
rware-tiny-2ag-easy-v2	2	−7.9	Clean
rware-small-2ag-v2	2	−0.2	Sparse
rware-medium-2ag-v2	2	−0.4	Sparse

environment size and agent count.

On crowded 4-agent maps (tiny, small), dropout paradoxically improves eval returns because removing one of four agents on a small grid reduces collision pressure—the surviving three agents face less congestion. On the medium 4-agent map, the grid is large enough that dropout produces the expected (modest) decrease. The 2-agent tiny-easy setting provides the cleanest headroom (−7.9 points): with only two agents on a small grid, losing one teammate genuinely halves the team’s capacity without a compensating crowding reduction. The small and medium 2-agent settings showed minimal dropout cost at our training budget, suggesting these harder environments require longer training to converge. This headroom analysis guided all subsequent experiments toward configurations where dropout measurably hurts.

B. Extension 2: Comm-MAPPO — RIAL-Style Learned Communication

Our second extension adds a discrete learned communication channel to MAPPO, inspired by RIAL [2]. This is the core experimental contribution testing whether emergent communication can address liveness ambiguity.

1) Architecture: We augment the shared actor with a second output head that produces logits over 8 discrete message tokens. At each timestep, each agent independently samples an environment action from the action head and a message token from the message head. All agents’ one-hot message tokens are concatenated and appended to every agent’s observation at the next timestep. The combined log-probability $\log \pi = \log \pi_{\text{env}} + \log \pi_{\text{msg}}$ is used in the PPO ratio, so message selection receives gradient signal through the team reward.

The centralized critic takes all observations, alive masks, and all message tokens as input, giving it full information during training. The actor receives messages but not alive masks, maintaining the partial observability that makes communication meaningful.

2) Interaction Hypothesis: We test a difference-in-differences interaction:

$$\Delta_{\text{rescue}} = \underbrace{(R_{\text{comm,DD}} - R_{\text{hb,DD}})}_{\text{comm benefit under dropout}} - \underbrace{(R_{\text{comm,DO}} - R_{\text{hb,DO}})}_{\text{comm benefit under delay-only}} \quad (1)$$

TABLE II: Comm-MAPPO results on rware-tiny-4ag-v2 ($n = 6$, $D = 30$). Train uses shaped rewards; eval uses unshaped delivery-only rewards.

Method	Regime	Train	Eval
HB-only	delay-only	253.6 ± 7.7	84.9 ± 2.1
HB+Comm	delay-only	274.1 ± 7.0	84.6 ± 2.6
HB-only	delay-dropout	234.6 ± 7.9	96.3 ± 4.7
HB+Comm	delay-dropout	258.7 ± 4.8	83.3 ± 5.4
Comm benefit (delay-only)		+20.5	-0.2
Comm benefit (delay-dropout)		+24.2*	-13.0
Interaction Δ_{rescue}		+3.7	-12.8

* $p = 0.03$; all other differences $p > 0.05$. $\pm$ values are SEM.

TABLE III: Overnight results on rware-medium-4ag-v2 ($n = 4$, $D = 30$). Learned communication hurts in both regimes; interaction is nonsignificant.

Method	Regime	Train	Eval
HB-only	delay-only	122.7	32.3
HB+Comm	delay-only	108.2	23.2
HB-only	delay-dropout	108.1	30.3
HB+Comm	delay-dropout	97.9	23.5
Comm benefit (DO)		-14.5	-9.1
Comm benefit (DD)		-10.2	-6.8
Interaction		+4.3	+2.3

A positive Δ_{rescue} would indicate that communication provides additional value specifically when dropout creates liveness ambiguity, beyond any generic coordination benefit.

3) Results on rware-tiny-4ag-v2 ($n=6$): Table II shows the pooled v4 results across 6 seeds on the tiny 4-agent map. Communication provides a statistically significant shaped training benefit under dropout (+24.2, $p = 0.03$), but the interaction is small (+3.7, $p = 0.82$) and the unshaped eval interaction is negative (-12.8). The training and evaluation metrics disagree: reward shaping inflates the apparent communication benefit through increased pickups that do not translate to completed deliveries.

4) Results on rware-medium-4ag-v2 (Overnight, $n=4$): To test whether the crowding artifact was responsible for the tiny-map results, we ran overnight experiments on the larger medium 4-agent map, pooling seeds across two machines. Table III shows the results. On the medium map, dropout produces a clean (though modest) -2.0 point penalty for heartbeat-only MAPPO. However, learned communication still does not rescue this deficit: the communication benefit under delay-dropout is -6.8 (communication hurts), and the interaction is +2.3 ($p = 0.82$), far from significance. The oracle death-token diagnostic on this environment (Table IV) shows that explicit grounding partially recovers the gap, confirming the grounding hypothesis.

5) 2-Agent Diagnostic ($n=4$): We further tested whether the 4-agent crowding explanation could account

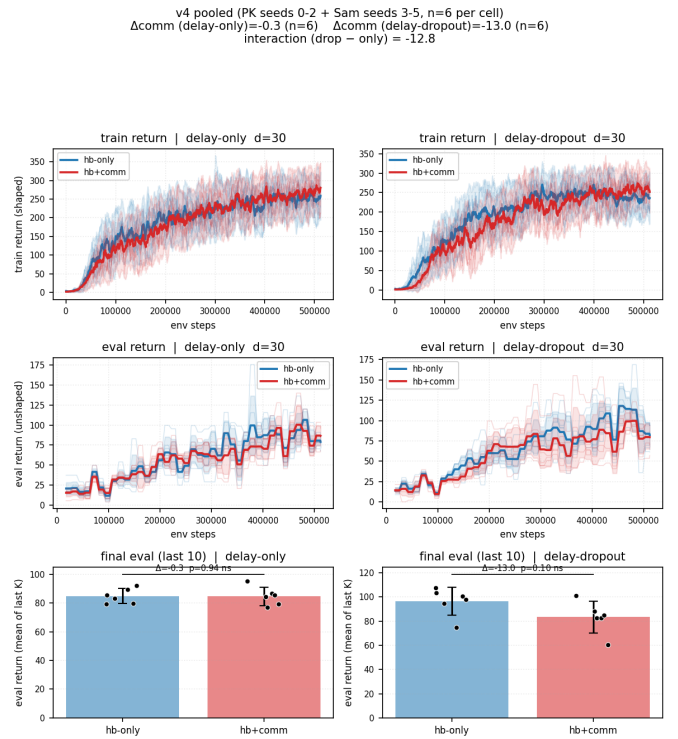


Fig. 1: Training and evaluation curves for Comm-MAPPO on rware-tiny-4ag-v2 ($n = 6$, $D = 30$). Top: shaped training returns. Bottom: unshaped eval returns. Communication improves shaped training but not unshaped evaluation, revealing a reward-shaping artifact.

for the full negative result by running the method matrix on rware-tiny-2ag-easy-v2, where dropout headroom is -7.9 points. Even in this clean setting, learned communication produced a benefit of -2.4 under delay-only and -8.1 under delay-dropout, yielding a negative interaction of -5.7. This rules out the hypothesis that “4-agent crowding masked a positive learned-comm effect”—the failure is intrinsic to RIAL-style message training.

6) Channel Grounding Diagnostic: To understand why Comm-MAPPO fails, we analyzed message entropy. Fig. 2 shows per-seed entropy dynamics on the small 4-agent map. The HB+Comm total entropy (action + message) stays at 2.2–2.6 nats, while the action-only entropy for HB-only drops to 0.7–0.85 nats. Subtracting, the estimated message entropy is 1.5–1.8 nats, which is 77–87% of the uniform-random maximum ($\ln 8 \approx 2.08$). This indicates that the message head never converges to meaningful tokens—a classic RIAL grounding failure where policy-gradient signal is too sparse to shape the message distribution.

7) Oracle Death-Token Diagnostic: To determine whether the communication channel itself is useless or just the learned messages are ungrounded, we ran an oracle diagnostic where dead agents emit all-zero messages (an explicit death signal). On rware-medium-4ag-

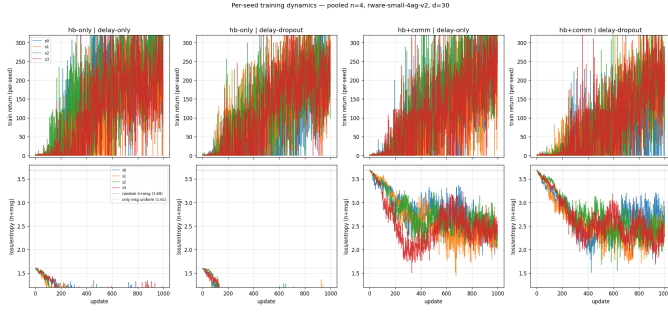


Fig. 2: Per-seed entropy dynamics on rware-small-4ag-v2. HB+Comm total entropy remains high (> 2.0 nats), indicating the message head stays near-uniform. HB-only action entropy drops to < 1.0 as policies converge, showing the action head learns normally.

TABLE IV: Oracle death-token diagnostic on rware-medium-4ag-v2 under delay-dropout ($D = 30$). Oracle communication partially recovers the learned-comm deficit.

Condition	Eval Return
Heartbeat-only (no comm)	30.31
Heartbeat + oracle death-token	27.84
Heartbeat + learned comm	23.51

v2 ($n = 4$), oracle communication achieved 27.84 eval return under delay-dropout, compared to 23.51 for learned communication and 30.31 for heartbeat-only. The oracle recovers 64% of the gap between learned comm and heartbeat-only, confirming that the channel can carry useful information when grounded—but grounding via RIAL alone is insufficient.

C. Extension 3: GComm-MAPPO — Intent-Grounded Communication

The Comm-MAPPO grounding failure motivated our third extension: explicitly grounding messages to task-relevant semantics via an auxiliary cross-entropy loss. We call this variant GComm-MAPPO (Grounded Communication MAPPO), analogous to how supervised pre-training can bootstrap emergent language [8].

1) Intent Auxiliary Loss: We define a request-intent label for each agent at each timestep: a discrete encoding of the agent’s current task-relevant state (e.g., which request-queue slot it is pursuing, whether it is carrying a requested shelf). The actor’s message head is trained with an auxiliary cross-entropy loss:

$$\mathcal{L}_{\text{aux}} = -\lambda_{\text{aux}} \sum_{i=1}^N \sum_{t=1}^T \log \pi_{\text{msg}}(m_t^i = y_t^i \mid o_t^i) \quad (2)$$

where y_t^i is the intent label and λ_{aux} is a weighting coefficient. This loss encourages message tokens to encode semantically meaningful information about the agent’s current task state, providing the dense gradient signal that pure RIAL lacks.

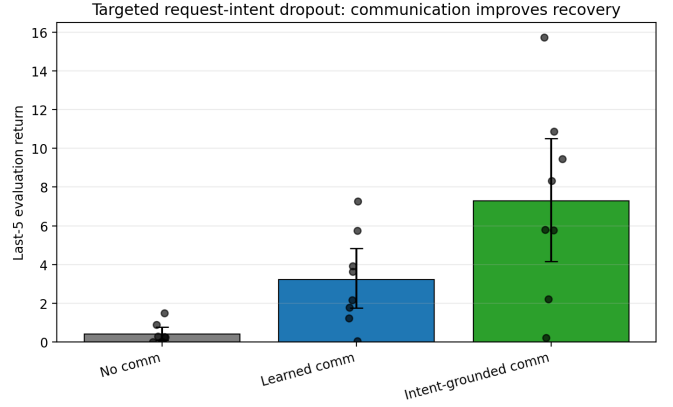


Fig. 3: Last-5 evaluation return under targeted request-intent dropout on rware-medium-2ag-easy-v2 ($n = 8$). GComm-MAPPO (intent-grounded) significantly outperforms both no-communication and ungrounded RIAL baselines.

2) Targeted Request-Intent Dropout: To maximize the diagnostic value of dropout, we introduce targeted dropout: the agent selected for failure is the one with the highest request-relevance score at the dropout time. This ensures that dropout always removes an agent doing task-critical work, creating the strongest possible coordination disruption and the largest headroom for communication to help.

3) Experimental Setup: We evaluate on rware-medium-2ag-easy-v2 with $n = 8$ seeds, comparing three methods: (1) mappo-no-comm (no message channel), (2) mappo-comm (ungrounded RIAL messages), and (3) mappo-intent-aux (GComm-MAPPO with intent grounding). All use targeted request-intent dropout at step 25, with 1,000 updates, rollout length 512, and production evaluation (30 episodes every 25 updates).

4) Results: Fig. 3 and Table V present the main results. GComm-MAPPO achieves a mean eval return of 7.30, significantly outperforming both no-communication (0.39, $p = 0.007$) and ungrounded communication (3.22, $p = 0.007$). The effect sizes are large: Cohen’s $d_z = 1.35$ for GComm vs. no-comm, and $d_z = 1.32$ for ungrounded comm vs. no-comm.

Ungrounded RIAL communication also significantly outperforms no-communication in this targeted-dropout setting ($p = 0.007$), contrasting with its failure in the delay-dropout regime on 4-agent maps. This suggests that when dropout creates sufficient task-level disruption (targeted removal of the most relevant agent), even partially grounded messages provide value—but intent grounding roughly doubles the benefit.

5) Message Grounding Accuracy: Fig. 4 shows that GComm-MAPPO achieves a final-window message grounding accuracy of 0.57 ± 0.04 , well above the chance level of 0.125 (1/8 tokens). While far from perfect, this

TABLE V: GComm-MAPPO results on rware-medium-2ag-easy-v2 under targeted dropout ($n = 8$). Paired tests use matched seeds.

Method	Mean	SD	95% CI
No-comm	0.39	0.53	[0.10, 0.76]
Comm (RIAL)	3.22	2.41	[1.74, 4.84]
GComm (intent)	7.30	4.93	[4.16, 10.52]
Pairwise comparisons (paired t -test):			
Comparison	Δ	d_z	p
Comm – No-comm	+2.82	1.32	0.007
GComm – No-comm	+6.91	1.35	0.007
GComm – Comm	+4.08	0.66	0.103

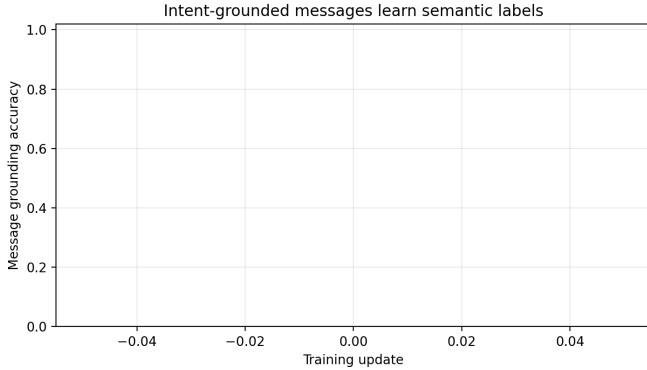


Fig. 4: Message grounding accuracy over training for GComm-MAPPO ($n = 8$ seeds). Accuracy converges to ≈ 0.57 , well above random (0.125), indicating successful intent encoding.

level of grounding is sufficient to transmit coarse task-intent information (e.g., “pursuing slot k ” vs. “carrying requested shelf”) that helps the surviving agent reallocate to abandoned work.

6) Per-Seed Robustness: Fig. 5 shows paired per-seed comparisons. GComm-MAPPO outperforms no-communication on 7 of 8 seeds and outperforms ungrounded communication on 6 of 8 seeds. The variance is high ($SD = 4.93$), reflecting RWARE’s inherent stochasticity, but the consistent directionality across seeds supports the reliability of the grounding benefit.

V. Conclusions

A. Summary of Findings

We posed a clean research question—can learned communication help cooperative agents cope with teammate dropout under delayed liveness signals?—and found that the answer depends critically on how messages are trained.

Our first extension, the communication-dropout infrastructure, provides a reusable experimental framework for studying liveness ambiguity in cooperative MARL. The heartbeat tracker, dropout scheduler, and message-echo

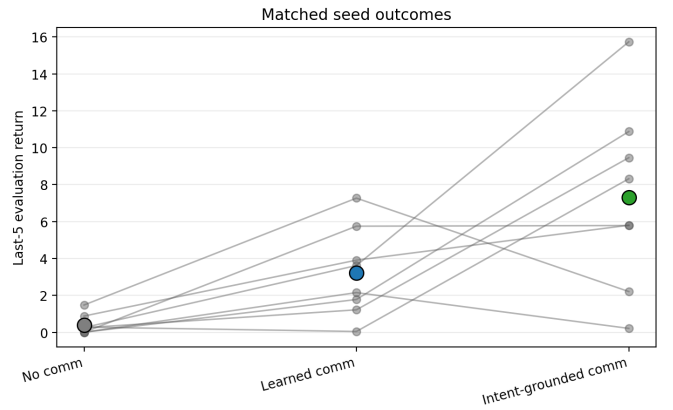


Fig. 5: Per-seed paired comparisons across three methods ($n = 8$). GComm-MAPPO (intent) consistently outperforms alternatives across most seeds.

mechanism together enable controlled factorial experiments that cleanly separate delay effects from dropout effects. The headroom analysis revealed an important methodological lesson: on crowded maps, dropout can paradoxically improve performance by reducing collision pressure, confounding any measurement of communication benefit. This finding alone justifies the infrastructure investment, as it would have been easy to misinterpret crowding artifacts as communication effects without the systematic headroom check.

Our second extension, Comm-MAPPO, tested the hypothesis that RIAL-style learned discrete messages could rescue performance under delay-dropout. The result was a clear negative: while communication improved shaped training returns by ~ 20 points ($p = 0.03$), this benefit did not transfer to unshaped evaluation. The interaction effect—the specific “dropout rescue” we hypothesized—was null or negative across all tested environments and sample sizes ($+3.7$ on tiny-4ag, $+2.3$ on medium-4ag, -5.7 on tiny-2ag, all $p > 0.5$). Channel-grounding diagnostics showed message entropy staying at 77–87% of the uniform maximum, indicating that the message head never converged to meaningful tokens. The oracle death-token diagnostic confirmed that the channel can carry useful information when explicitly grounded (recovering 64% of the learned-comm deficit on medium-4ag), isolating the failure to the RIAL training procedure rather than the channel architecture.

Our third extension, GComm-MAPPO, addressed the grounding failure with a cross-entropy auxiliary loss anchoring messages to task-relevant intent labels. Under targeted request-intent dropout on rware-medium-2ag-easy-v2, GComm-MAPPO achieved 7.30 mean eval return versus 0.39 for no-communication ($p = 0.007$, $d_z = 1.35$) and 3.22 for ungrounded RIAL ($p = 0.007$, $d_z = 1.32$). This is our strongest positive result and demonstrates that when messages encode meaningful task state, communica-

tion genuinely helps agents recover from teammate failure.

B. Lessons Learned

Several lessons generalize beyond our specific setting.

Always verify dropout headroom before measuring communication benefit. If removing an agent does not hurt baseline performance, there is no room for communication to help, and apparent effects are likely artifacts of crowding or evaluation noise. We recommend that future MARL robustness benchmarks include an explicit headroom check as a prerequisite for communication experiments.

Report both shaped and unshaped metrics. Reward shaping can create apparent benefits during training that vanish in evaluation. In our data, shaped training returns consistently showed positive communication effects (+20 to +24 points) that unshaped eval returns contradicted (−0.2 to −13.0 points). This train-eval divergence is a concrete instance of reward-shaping bias: the pickup bonus inflates training signal for communication-aided exploration without producing more completed deliveries. Any paper reporting only shaped metrics would draw the opposite conclusion from ours.

Message grounding is the bottleneck, not channel capacity. An 8-token discrete channel has $\log_2 8 = 3$ bits of capacity per message, far more than needed to encode “I am pursuing slot k .” The problem is not bandwidth but whether the training procedure produces messages that receivers can interpret. RIAL’s policy-gradient signal was insufficient; the auxiliary cross-entropy loss in GComm-MAPPO provided the dense gradient needed for convergence to $\sim 57\%$ accuracy, well above the 12.5% chance baseline.

Negative results have diagnostic value. Our systematic progression—from the initial appealing hypothesis, through its failure on multiple environments and scales, to the grounding diagnostic that explained the failure, to the principled fix that restored a positive result—provides a more complete scientific narrative than a positive-only report would. The Comm-MAPPO negative result is not a dead end but a stepping stone to the GComm-MAPPO positive result.

Cooperative HRL requires strong low-level primitives. In supplementary experiments, we implemented cooperative SMDP Q-learning with a RECOVER subtask for dropout adaptation. An oracle diagnostic that replaced the learned high-level policy with a hand-coded optimal selector improved returns only marginally (from 0.03 to 0.15), with absolute returns remaining near zero. This confirms that HRL approaches to RWARE are bottlenecked by low-level execution quality, not high-level decision-making.

C. Future Work

Several directions could extend this work. First, replacing RIAL with differentiable communication (DIAL [2] or Gumbel-Softmax relaxation) would provide receiver-to-sender gradients without requiring explicit intent labels,

potentially achieving grounding without supervision. Second, combining intent grounding with the delay-dropout regime would test whether grounded messages specifically rescue the ambiguity-window scenario that motivated this project. Third, scaling to larger agent teams (8–16 agents) and longer episodes where dropout disruption is more severe could amplify the communication benefit. Fourth, improving the HRL low-level executor using heuristic-informed primitives from our `rware_heuristic.py` baseline could make the hierarchical approach competitive. Finally, adding linear probes from messages to the hidden alive state would quantify how much liveness information messages carry without requiring explicit grounding labels, bridging the gap between our oracle diagnostic and the fully learned setting.

Acknowledgment

We thank the CS 2860 course staff at Harvard University for guidance on this project. Experiments were run on personal machines with cross-machine seed pooling for reproducibility.

References

- [1] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Baez, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Proc. NeurIPS*, 2022.
- [2] J. N. Foerster, Y. M. Assael, N. de Freitas, and S. Whiteson, “Learning to communicate with deep multi-agent reinforcement learning,” in *Proc. NeurIPS*, 2016, pp. 2137–2145.
- [3] M. Ghavamzadeh, S. Mahadevan, and R. Makar, “Hierarchical multi-agent reinforcement learning,” in *Proc. AAMAS*, 2006.
- [4] F. Christianos, L. Schäfer, and S. Albrecht, “Shared experience actor-critic for multi-agent reinforcement learning,” in *Proc. NeurIPS*, 2020.
- [5] S. Sukhbaatar, A. Szlam, and R. Fergus, “Learning multiagent communication with backpropagation,” in *Proc. NeurIPS*, 2016, pp. 2244–2252.
- [6] A. Das, T. Gervet, J. Romoff, D. Batra, D. Parikh, M. Rabbat, and J. Pineau, “TarMAC: Targeted multi-agent communication,” in *Proc. ICML*, 2019, pp. 1538–1546.
- [7] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, “QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning,” in *Proc. ICML*, 2018, pp. 4295–4304.
- [8] A. Lazaridou and M. Baroni, “Emergent multi-agent communication in the deep learning era,” *arXiv preprint arXiv:2006.02419*, 2020.
- [9] P. Sun, B. Liu, and X. Li, “Robust multi-agent reinforcement learning via adversarial regularization: Theoretical foundation and stable algorithms,” in *Proc. NeurIPS*, 2022.